

Challenge: Inventory Reservation System

Overview

In a high-traffic "Flash Sale" environment, the difference between a successful reservation and a "double-sell" is a few milliseconds and a robust concurrency strategy.

Your task is to build a **Stock Reservation System** that allows users to hold items for a limited window. You must demonstrate mastery of the [Spec Kit](#) workflow, ensuring that your architectural decisions are documented, validated, and refined before a single line of application code is written.

The Spec Kit Workflow (Strictly Enforced)

Your repository must reflect the SpecKit framework's sequence of artifacts. We will evaluate the Git history to ensure the "Architecture First" approach was followed.

Functional Requirements

- **Inventory Dashboard:** Display items with their current "Reserved" and "Available" (Total - Reserved) counts.
- **Atomic Reservations:** Users can reserve N units of an item. The system must prevent over-reservation even under high concurrent load.
- **Reservation TTL:** Reservations must automatically expire after **60 seconds** if not "confirmed". Upon expiration, the reservation is permanently removed and cannot be interacted with, and the stock is returned to the "Available" pool.
- **Manual Release:** Users should be able to manually cancel/release their reservation at any time, including after the TTL has elapsed (e.g., if the UI timer is out of sync)
- **Conflict Handling:** The UI must gracefully handle and display errors when a user tries to reserve an item that was *just* taken by someone else.
- **OpenAPI Contract:** Provide the Open API contract for the REST API implemented.

Idempotency Requirement

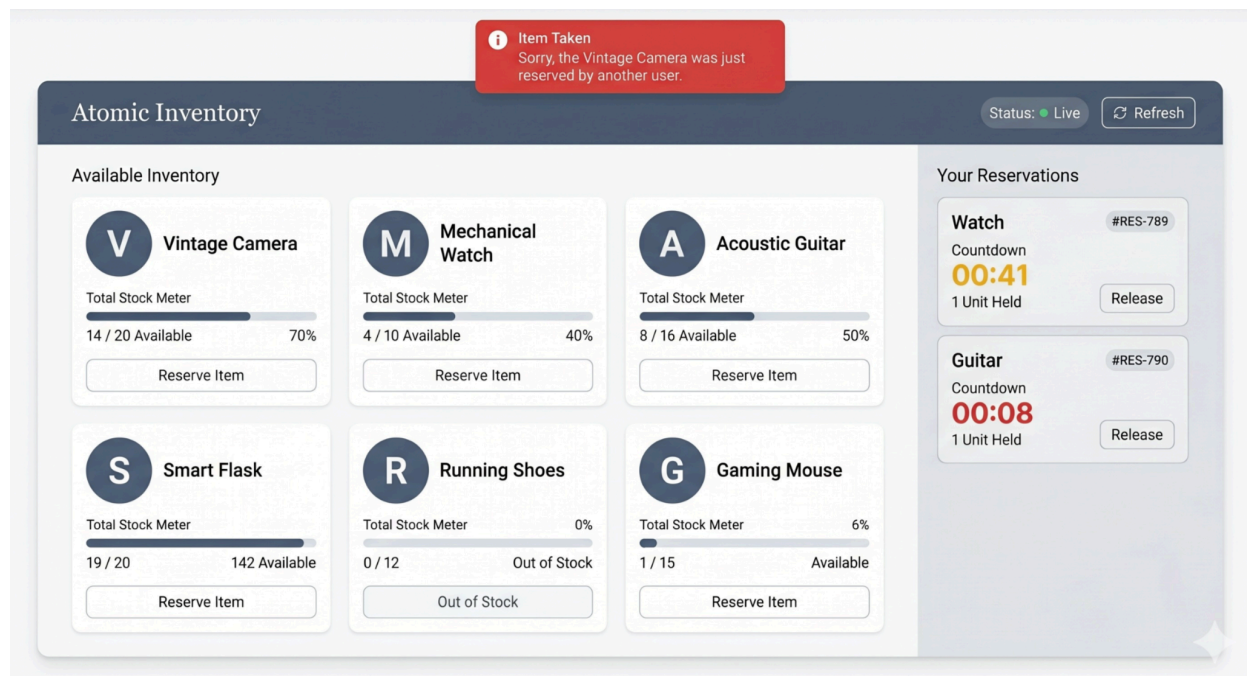
Both reserve and release must be idempotent. Network retries, double-clicks, and at-least-once delivery from the frontend must not produce duplicate state changes.

- `POST /reservations` must accept an Idempotency-Key header. Two requests with the same key and the same payload must return the same reservation (same ID, same outcome) and must not decrement stock twice. Two requests with the same key but different payloads must be rejected with a clear error.

- **DELETE /reservations/:id** must be safe to call repeatedly. Releasing an already-released reservation must succeed (or return a well-defined no-op response) and must not return stock units to the pool more than once.

UI/UX Requirements

- **Layout:** A list view with item name, total stock, available stock, and a reserve action.
- **User Feedback:** Visible feedback for reserve success, validation errors (insufficient stock, invalid quantity), and release.
- **Status:** A view of the user's active reservations with a release button.
- **Misc:** Loading and error states.
- **Visual Reference:** Use this reference as the design guideline for the Frontend.



Constraints

- **Time Budget:** ~8–10 hours of focused work. Submit what you have at 12 hours; do not exceed.
- **Backend:** Go (standard library or lightweight router like Chi/Gin). Using their latest versions.
- **Database:** Use a PostgreSQL database. You must handle race conditions.
- **Frontend:** React + Vite + TypeScript (using their latest versions). Focus on clean component composition and handling "loading/error" states for asynchronous actions.
- **Testing:**
 - **Go:**

- i. A concurrency test that simulates 50+ simultaneous reservation requests for the last remaining item.
 - ii. A test where 100 concurrent reserve requests for 10 units must result in exactly 10 successful reservations and 90 rejections, with no negative stock.
 - iii. An idempotency test for `reserve`, the same `Idempotency-Key` sent twice in parallel results in one reservation and one stock decrement.
 - iv. An idempotency test for `release`, calling release twice on the same reservation results in stock returned exactly once.
- **React:**
 - i. Unit tests for the reservation timer logic.
 - ii. At least one component test covering the reserve flow happy path and one covering an error state (e.g., insufficient stock)

Evaluation Rubric

Criteria	Expected Results	Insufficient
Spec Kit Rigor	<code>spec.md</code> identifies and resolves edge cases before coding.	Artifacts are complete but mostly describe the "happy path."
Concurrency	Correct use of PostgreSQL and/or Go to prevent race conditions. Zero over-selling in load tests.	Functional under normal load, but prone to race conditions under high concurrency.
State Management	React UI stays perfectly in sync with the backend; handles "Expired" states via WebSockets or polling.	UI requires manual refreshes to show updated stock levels.
Traceability	The final code maps 1:1 to the <code>tasks.md</code> . Commits are clean and follow the workflow.	Tasks are vague; code deviates significantly from the initial <code>plan.md</code> .
Frontend Implementation	The frontend is aligned to the provided reference and pass usability tests	The frontend is somewhat aligned with the reference but doesn't pass usability tests

Submission Artifacts

Provide the URL to a public repository of yours with the following:

1. Full chat history of the conversation with the chosen LLM used for this challenge.
2. Everything generated by SpecKit (without exception).
3. Go Backend Source Code.
4. React Frontend Source Code.
5. PostgreSQL seed data for reviewing purposes.
6. A [README.md](#) explaining the concurrency strategy, how to run the test suite and the LLM used for the project (and why).
7. A [spec-kit-notes.md](#) explaining commands used, assumptions and refinements along with any "Pivots" you made (e.g., "I realized during the Plan phase that X wouldn't work, so I changed to Y").

Note: There might be intentional ambiguity in this challenge. Please document your assumptions or ask for clarification when applicable.